

计算机网络实验报告三-2 滑动窗口可靠传输协议

学号:2013365 姓名:颜铭

1. 实验准备

1.1 实验要求

利用数据报套接字在用户空间实现面向连接的可靠数据传输

包含以下基本功能为：

- 建立连接
- 差错校验和检测
- 不同接收情况的处理，拥塞控制
- 超时重传，丢包重传

本次实验在上次流量控制算法上引入滑动窗口机制以及建立了更完备的拥塞控制算法来验证累积确认快速重发的效果

1.2 前置编程

本次实验使用路由程序作为模拟丢包和设置延时的转发应用，程序转包最大不超过15000bytes。此外注意先启动路由程序才可以沟通服务器和客户端，需要将路由器绑定都可用的 `0.0.0.0::<port>` 端口，设置防火墙规则。

1. 需要在IP和端口设计中需要将客户端 `bind` 到客户端的客户IP (`任意一个非路由和服务器端口`)，并且通过这个端口接收消息。服务端同理，将服务端的接收方绑定为服务器IP。无论是客户端还是服务器都要将发送的目标对象的IP设置为路由程序的IP。
2. 服务端使用windows的 `WSA` 套接字编程，引入 `ioctlsocket` 设置服务器为非阻塞监听，对 `ANY` 端口的响应进行实时的监听。客户端作为发送方才可以模拟丢包，因此需要让客户端主动发起握手请求后多一轮确认服务器的握手成功准备发送数据的通信。
3. 需要重新设计 `struct packet`，至少包含序列号，校验和，ACK确认以及不同的通信编码标识（可以等价与FIN，通过不同tag）。

2. 实验原理和设计

参考TCP在不可靠的网络层服务之上实现可靠传输的各种机制，在应用层添加这些机制，以实现在传输层中不可靠的UDP服务之上提供可靠传输

2.0.1 对于TCP的握手原理

- **第一次握手** 客户端将TCP报文标志位SYN置为1，按通信序列产生一个序号值 $seq = x$ ，保存在TCP首部的序列号(Sequence Number)字段里，指明客户端打算连接的服务器的端口，并将该数据包发送给服务器端，发送完毕后，客户端进入 SYN_SENT 状态，等待服务器端确认。
- **第二次握手** 服务器端收到数据包后由标志位SYN=1知道客户端请求建立连接，服务器端将TCP报文标志位SYN和ACK都置为1， $ack = x + 1$ ，随机产生一个序号值 $seq = K$ ，并将该数据包发送给客户端以确认连接请求，服务器端进入 SYN_RCVD 状态。
- **第三次握手** 客户端收到确认后，检查ack是否为 $x + 1$ ，ACK是否为1，如果正确则将标志位ACK置为1， $ack = K + 1$ ，并将该数据包发送给服务器端，服务器端检查ack是否为 $K + 1$ ，ACK是否为1，如果正确则连接建立成功，客户端和服务器端进入 ESTABLISHED 状态，完成三次握手，随后客户端与服务器端之间可以开始传输数据了。

在确认应答的时候TCP会携带Seq，接收到该报文段的另一端发送确认报文回去，报文中包含确认号($ack=seq+1$)，为期待接收的下一字节的序号。这样也可以维护发送的报文信息有序

2.0.2 TCP的报文结构

我们主要需要维护单向通信的建立连接和分发数据包信息。以及数据段的长度和以char为元单位的内容(我们在读取文本和图形的二进制数据时可以需要通过字节流进行内存写入)

我们关注的原理性结构为:

序列号(Seq): 长度为32 bits (4个字节)。指定了当前数据分片中分配给第一字节数据的序列号。在TCP传输流中每一个字节为一个序号。如果TCP报文中flags标志位为SYN，该序列号表示 初始化序列号(ISN)，此时第一个数据应该从序列号ISN+1开始。

确认序列号 (Ack): 长度为32bits (4个字节)。表示TCP发送者期望接受下一个数据分片的序列号。该序号在TCP分片中Flags标志位为ACK时生效。序列号分片的方向和流的方向同 方向，而确认序列号分片方向和流方向反方向。

标志位(tag): 长度为6bits。表示TCP包特定的连接状态。一个标志位占一个bit，从低位到高位值依次为256内的不同状态码，表示不同的请求和应答状态。

校验和 (Checksum): 长度16bits (2个字节)。用来检查TCP头在传输中是否被修改。

数据 (Data): 长度可变。用来存储上层协议的数据信息,需要设计一个最大的数据存储长度。

窗口 (Window): 长度16bits (2个字节)。表示滑动窗口的大小，用来告诉发送端接收端的buffer space的大小。接收端buffer大小用来控制发送端的发送数据数率，从而达到流量控制。最大值为 65535(后置实验中使用)

2.0.3 滑动窗口机制的内容

GBN (Go-Back-N) 是一种可靠传输协议，通常用于在网络通信中的数据链路层。它是一种基于窗口的流水线协议，通过使用滑动窗口的概念来提高传输效率。下面是GBN算法的基本原理：

1. **滑动窗口：** GBN使用滑动窗口机制，允许一次性发送多个数据帧而无需等待对方的确认。发送方维护一个窗口，窗口内的帧可以被发送。接收方维护一个期望接收的帧的序号。
2. **序号：** 每个发送的帧都有一个序号，用于标识帧的顺序。序号通常是循环的，即达到最大值后重新从0开始。
3. **发送端操作：**
 - 发送方维护一个发送窗口，窗口内的帧都可以被发送。
 - 发送方发送窗口内的帧，并等待确认。
 - 一旦收到确认，发送方将窗口向前滑动，准备发送下一批帧。
4. **接收端操作：**
 - 接收方维护一个期望接收的帧的序号，即期望序号。
 - 当接收到帧时，检查其序号。
 - 如果序号等于期望序号，则接收帧，并发送确认。
 - 如果序号不等于期望序号，则丢弃帧，并发送对上一次成功接收的帧的确认。
 - 接收方向前滑动窗口，更新期望序号。
5. **超时重传：** 如果发送方在超时时间内未收到确认，它将重新发送窗口内的所有帧。接收方收到重复的帧时，将丢弃重复的帧并发送对上一次成功接收的帧的确认。
6. **选择性重传：** GBN协议通常使用累积确认，即一次确认所有已成功接收的帧。因此，当发送方收到超时重传的确认时，它将知道需要重传的是第一个未被确认的帧，而之前的帧都已成功接收。

2.1 实验设计

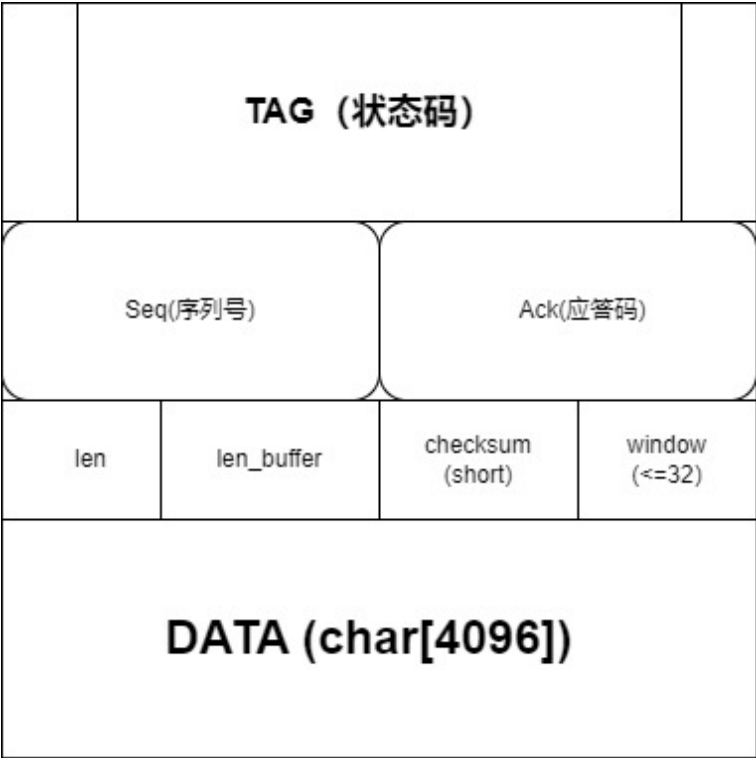
实验设计客户端的端口为 88881, 路由程序的端口为 1099, 服务器端的端口为 8888。三者皆为本地回环IP地址 127.0.0.1

对于数据包，考虑到与之后的实验兼容要加上端口和预留数据信息。设计为：

```
#define MESSAGELEN 4096
struct packet
{
    unsigned char tag; //连接建立、断开标识
    unsigned int seq; //序列号
    unsigned int ack; //确认号
    unsigned short len; //数据部分长度
    unsigned short len_buffer; //预留数据段位置长度信息。
    unsigned short checksum; //校验和
    unsigned short window; //窗口(停等机制不需要使用)
    char data[MESSAGELEN]; //数据长度
}
```

这里的 seq 和 window 在停等试验中都不是必要的。然后需要确认ack和tag标识的作用。

同理得到协议设计的架构图为：



校验和算法为：

```
unsigned short make_sum(int count, char* buf)
{
    unsigned long sum = 0;

    while (count)
    {
        sum += *buf;

        if (sum & 0xffff0000)
        {
            sum &= 0xffff;
            sum++;
        }

        return ~(sum & 0xffff);
    }
}
```

Fence 2-2

检验部分为：

```
bool corrupt(packet* pkt)
{

```

```

int count = sizeof(pkt->data) / 2;
register unsigned long sum = 0;
unsigned short* buf = (unsigned short*)(pkt->data);
while (count--) {
    sum += *buf++;
    if (sum & 0xFFFF0000) {
        sum &= 0xFFFF;
        sum++;
    }
}
if (pkt->checksum == ~(sum & 0xFFFF))
    return true;
return false;
}

```

Fence 2-3

2.1.1 通信流程

- 握手，首先客户端发送 `tag==1` 的请求握手连接请求。服务端接收后应答100请求后准备接收 `tag==200` 表示允许接受数据。完成三次握手建立连接，然后客户端需要准备数据初始化，使用预留数据段发送文件名data。使得服务器端路径创建文件并打开文件流。
- 挥手，客户端完成发送后发送 `tag==88` 进行广播。服务器端应答188表示检查全部将接受无误后进入挥手阶段，服务器端发送202码，然后客户端接收后发送212码表示请确认关闭连接，服务器端再发送213码表示服务器端预关闭状态，客户端确认 `tag==213` 后进行本地关闭，清空套接字后总共四次挥手。

此外拥塞控制的基本单位实现为 `cwnd` 拥塞窗口和流量阈值 `ssthresh`

3. 实验代码实现

首先考虑实现超时重传机制，初始将ack都设置为1标识期望最小有序待定状态。然后对于返回接受的ack标记为 `ack==3` 而在前面发送时标记为 `ack==2`。超时重传就是在特定的时限后调用函数检查是否已有的单个窗口内序列的ack还是2则认为没有成功传递。在这之前避免占用窗口使用拥塞控制的快速重传实现对临界的ACK应答进行释放。

对于接受不同状态下带拥塞控制的确认，预先设计为慢启动状态。也即在很大的阈值范围内不超过单个串口的单个对ack确认后再触发全局的发送信号开关。

//快速重传

```
void FASTRECOhandler()
{
    packet* pkt1 = new packet;
    pkt1->init_packet();
    for (int i = curack; i != curseq; i = (i++) % seqnumber)
    {
        memcpy(pkt1, &buffer[i % WINDOWSIZE], BUFFER);
        sendto(socketClient, (char*)pkt1, BUFFER, 0, (SOCKADDR*)&addrServer, sizeof(SOCKADDR));
        cout << "[Repeat-Send " << i << " 号数据包]" << endl;
    }
}
```

Fence 3-1

//超时重传

```
void timeouthandler()
{
    BOOL flag = false;
    packet* pkt1 = new packet;
    pkt1->init_packet();
    if (ack[curack % WINDOWSIZE] == 2)//快速重传之后还有没被确认的，认为包丢失
    {
        for (int i = curack; i != curseq; i = (i++) % seqnumber)
        {
            memcpy(pkt1, &buffer[i % WINDOWSIZE], BUFFER);
            sendto(socketClient, (char*)pkt1, BUFFER, 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
            cout << "[Repeat-Send (TimeOut) " << i << " 号数据包]" << endl << endl;
            flag = true;
        }
    }
    if (flag == true)
    {
        ssthresh = cwnd / 2;
        cwnd = 1;
        STATE = SLOWSTART;//检测到超时，就回到慢启动状态
        cout << "[TIMEOUT]检测到超时，回到慢启动阶段" << endl;
        cout << "cwnd= " << cwnd << "      ssthresh= " << ssthresh << endl << endl;
    }
}
```

Fence 3-2

停等协议中由于curack和curseq的时序等效，因此不必超时重传。

以及客户端初始化程序:

//初始化工作

```
void init()
{
    //加载套接字库 (必须)
    WORD wVersionRequested;
    WSADATA wsaData;
    //套接字加载时错误提示
    int err;
    //版本 2.2
```

```

wVersionRequested = MAKEWORD(2, 2);
//加载 dll 文件 Socket 库
err = WSASStartup(wVersionRequested, &wsaData);
if (err != 0)
{
    //找不到 winsock.dll
    cout << "WSASStartup failed with error: " << err << endl;
    return;
}
if (LOBYTE(wsaData.wVersion) != 2 || HIBYTE(wsaData.wVersion) != 2)
{
    cout << "Could not find a usable version of Winsock.dll" << endl;
    WSACleanup();
}
else
{
    cout << "套接字创建成功" << endl;
}
socketClient = socket(AF_INET, SOCK_DGRAM, 0);

addrServer.sin_family = AF_INET;
addrServer.sin_port = htons(SERVER_PORT);
inet_pton(AF_INET, SERVER_IP, &addrServer.sin_addr);

//
addrClient.sin_family = AF_INET;
addrClient.sin_port = htons(CLIENT_PORT);
inet_pton(AF_INET, SERVER_IP, &addrClient.sin_addr);

err = bind(socketClient, (SOCKADDR*)&addrClient, sizeof(SOCKADDR));
if (err) {
    err = GetLastError();
    cout << "Could not bind the port" << SERVER_PORT << "for socket. Error code is"
<< err << endl;
    WSACleanup();
    return;
}
else
{
    cout << "客户端注册成功" << endl;
}

for (int i = 0; i < WINDOWSIZE; i++)
{
    ack[i] = 1; //初始标记为1表示最初的序列号
}
}

```

Fence 3-3

日志设计实现:发送和接受分别实现了对 `[Send]` 和 `[Recv]` 状态的检查。

GBN算法的实现是不断滑动窗口在出现丢包的时候重传整个可能出现传输失败的一整个窗口的数据包，并校准当前的ack，这可以在单个线程中通过while进行维护。

```
while ((curseq + seqnumber - curack) % seqnumber < minwindow(cwnd, WINDOWSIZE) &&
sendwindow>0)//只要窗口还没被用完，就持续发送数据包
{
    ZeroMemory(buffer[curseq % WINDOWSIZE], BUFFER);
    pkt->init_packet();
    if (length1 >= MESSAGELEN)
    {
        is.read(pkt->data, MESSAGELEN);
        make_pkt(pkt, curseq, MESSAGELEN);
        length1 -= MESSAGELEN;
    }
    else
    {
        is.read(pkt->data, length1);
        make_pkt(pkt, curseq, length1);
    }
    memcpy(buffer[curseq % WINDOWSIZE], pkt, BUFFER);
    sendto(socketClient, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrServer, sizeof(SOCKADDR));
    cout << "[Send 第 " << curseq << " 号数据包]" << endl << endl;
    ack[curseq % WINDOWSIZE] = 2;
    ++curseq;
    curseq %= seqnumber;
}
```

Fence 3-4

以及接受确认的实现为:

```
//等待接收确认
pkt->init_packet();
recvSize = recvfrom(socketClient, (char*)pkt, BUFFER, 0, ((SOCKADDR*)&addrClient), &len);
if (recvSize < 0)
{
    waitcount++;
    Sleep(200);
    if (waitcount > TIMEOUT / 0.2)
    {
        timeouthandler();
        waitcount = 0;
    }
}
else
{
    ackhandler(pkt->ack);
    auto use_time = clock() - st;
    cout << "ACK: " << pkt->ack << " CHECKSUM: " << pkt->checksum;
    cout<< " THROUGHPUT: " <<fixed<<setprecision(3)<< (double)(cur_bytes / use_time) << endl
<< endl;
    sendwindow = pkt->window;
}
break;
```

Fence 3-5

最后由握手设计原理的部分实现为

```
case 0://等待握手阶段
    recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&len);
    while (recvSize < 0)
    {
        pkt = new packet;
        pkt->init_packet();
        pkt->tag = 0;
        sendto(socketClient, (char*)pkt, BUFFER, 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
        Sleep(200);
        if (c1 % 100 == 0) {
            cout << "等待握手" << endl;
        }

        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0,
(SOCKADDR*)&addrClient, &len);
        c1++;
        if (recvSize >= 0)
        {
            c1 = 0;
            break;
        }
    }

    if (pkt->tag == 100)
    {

        pkt = connecthandler(200, totalpacket);
        cout << "服务器准备好接受数据" << endl << endl;
        memcpy(pkt->data, filepath, strlen(filepath));
        pkt->len = strlen(filepath);

        sendto(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
        stage = 1;

        break;
    }
}
```

Fence 3-6

挥手设计的代码实现为:

```
wave:{
    recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&length);
    int count_wave = 0;
    while(recvSize<0)
    {
        count_wave++;
    }
}
```

```

        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0,
(SOCKADDR*)&addrClient, &length);
        if(count_wave>100)
        {
            cout << "[Assertion]服务主机未知状态,可选择按下q键离开" << endl;

            char c;
            while (c = cin.get())
            {
                if (c == 'q')
                {
                    goto over;
                }
            }
        }

        if(pkt->tag==202)
        {
            pkt = connecthandlerClient(212);
            sendto(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrServer,
sizeof(SOCKADDR));
        }

        recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0, (SOCKADDR*)&addrClient,
&length);
        while(recvSize<0)
        {

            recvSize = recvfrom(socketClient, (char*)pkt, sizeof(*pkt), 0,
(SOCKADDR*)&addrClient, &length);

        }
        if(pkt->tag==213)
        {
            cout << "#####" << endl;
            cout << "挥手成功, 断开连接完成" << endl;
            cout << "#####" << endl;
            goto over;
        }

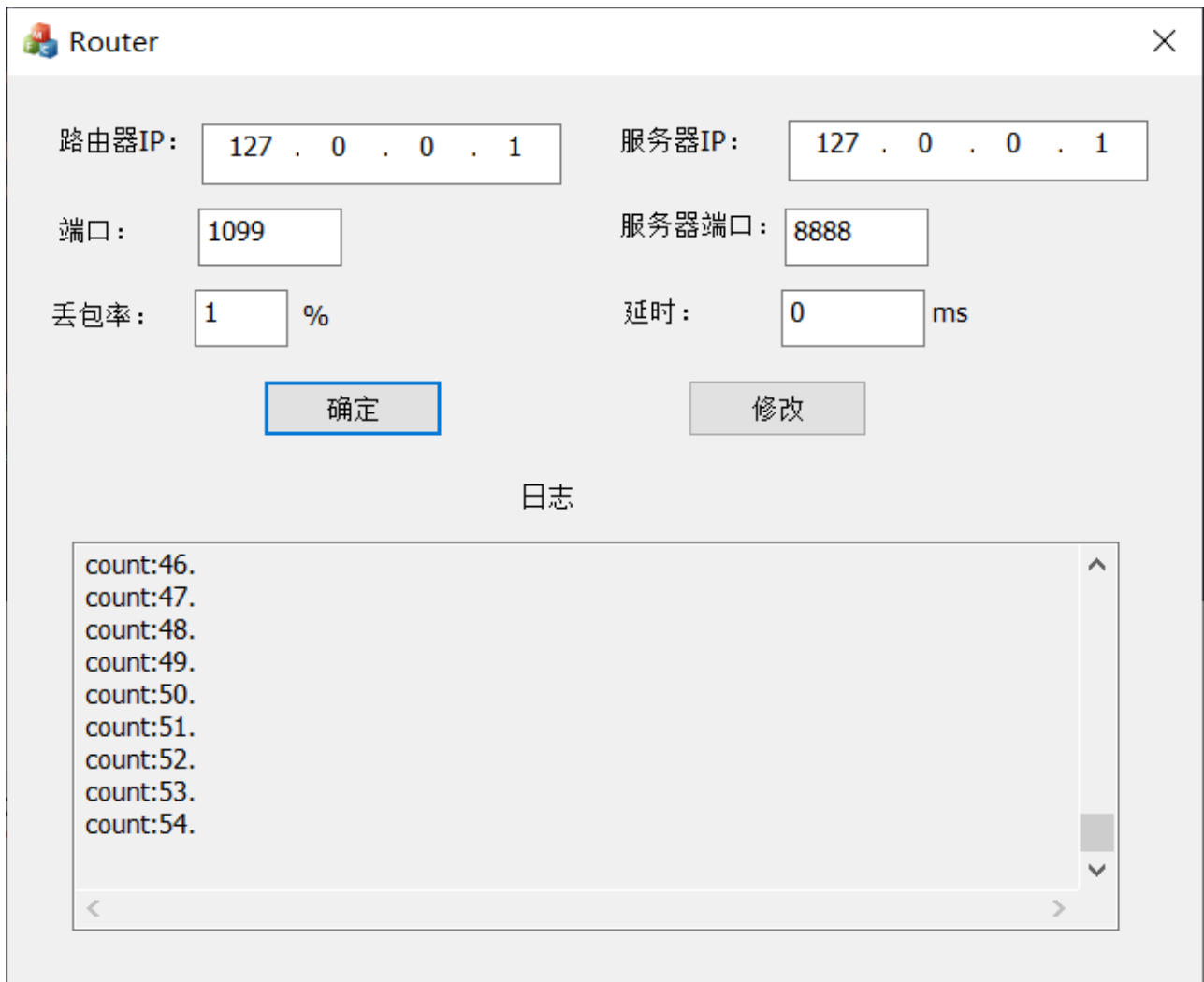
    }
    over:{
        closesocket(socketClient);
        WSACleanup();

        exit(0);
    }
}

```

4. 实验结果分析

路由程序的设置界面为



The image shows a window titled "Router" with a close button (X) in the top right corner. The window contains configuration fields for a router and a server. The fields are arranged in two columns. The left column contains: "路由器IP:" with a text box containing "127 . 0 . 0 . 1", "端口:" with a text box containing "1099", and "丢包率:" with a text box containing "1" followed by a "%" symbol. The right column contains: "服务器IP:" with a text box containing "127 . 0 . 0 . 1", "服务器端口:" with a text box containing "8888", and "延时:" with a text box containing "0" followed by "ms". Below these fields are two buttons: "确定" (OK) and "修改" (Modify). At the bottom of the window is a section titled "日志" (Log) containing a text area with the following text: "count:46.", "count:47.", "count:48.", "count:49.", "count:50.", "count:51.", "count:52.", "count:53.", "count:54.". The text area has a scrollbar on the right side.

Router

路由器IP: 127 . 0 . 0 . 1 服务器IP: 127 . 0 . 0 . 1

端口: 1099 服务器端口: 8888

丢包率: 1 % 延时: 0 ms

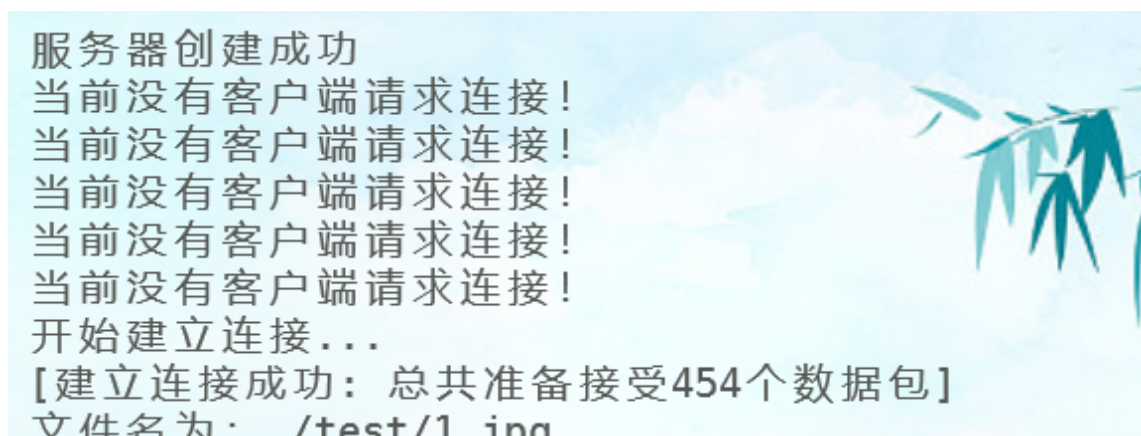
确定 修改

日志

count:46.
count:47.
count:48.
count:49.
count:50.
count:51.
count:52.
count:53.
count:54.

有程序运行结果截图为:

建立握手请求



The image shows a screenshot of a terminal window with a light blue background and a decorative bamboo branch on the right side. The text in the terminal is as follows: "服务器创建成功" (Server creation successful), "当前没有客户端请求连接!" (No client request connection at the moment!), "当前没有客户端请求连接!" (No client request connection at the moment!), "当前没有客户端请求连接!" (No client request connection at the moment!), "当前没有客户端请求连接!" (No client request connection at the moment!), "开始建立连接..." (Start establishing connection...), "[建立连接成功: 总共准备接受454个数据包]" ([Connection established: Total preparation to accept 454 data packets]), and "文件名为: /test/1 .img" (File name: /test/1 .img).

服务器创建成功
当前没有客户端请求连接!
当前没有客户端请求连接!
当前没有客户端请求连接!
当前没有客户端请求连接!
当前没有客户端请求连接!
开始建立连接...
[建立连接成功: 总共准备接受454个数据包]
文件名为: /test/1 .img

套接字创建成功
客户端注册成功
请输入要发送的文件名： ./test/1.jpg
文件大小为1857353Bytes,总共有454个数据包
等待握手
服务器准备好接受数据

服务器端

[Recv 第390号数据包]
[Send 第390号数据包的确认]
[Recv 第391号数据包]
[Send 第391号数据包的确认]
[GBN: 不是期待的数据包: 384 重发ack: 391]
[GBN: 不是期待的数据包: 385 重发ack: 391]
[GBN: 不是期待的数据包: 386 重发ack: 391]
[GBN: 不是期待的数据包: 387 重发ack: 391]
[GBN: 不是期待的数据包: 388 重发ack: 391]
[GBN: 不是期待的数据包: 389 重发ack: 391]
[GBN: 不是期待的数据包: 390 重发ack: 391]
[GBN: 不是期待的数据包: 391 重发ack: 391]
[GBN: 不是期待的数据包: 385 重发ack: 391]
[GBN: 不是期待的数据包: 394 重发ack: 391]
[GBN: 不是期待的数据包: 395 重发ack: 391]
[GBN: 不是期待的数据包: 396 重发ack: 391]
[GBN: 不是期待的数据包: 397 重发ack: 391]
[GBN: 不是期待的数据包: 398 重发ack: 391]

客户端

ACK: 33 CHECKSUM: 65494

[Send 第 34 号数据包](2023-11-14 12:44:11)

[Recv 第34号数据包的ACK确认](2023-11-14 12:44:11)

[慢启动阶段

cwnd= 36 sstresh= 1024]

ACK: 34 CHECKSUM: 106

[Send 第 35 号数据包](2023-11-14 12:44:11)

[Recv 第35号数据包的ACK确认](2023-11-14 12:44:11)

[慢启动阶段

cwnd= 37 sstresh= 1024]

ACK: 35 CHECKSUM: 65499

[Send 第 36 号数据包](2023-11-14 12:44:11)

[Recv 第138号数据包的ACK确认](2023-11-14 12:44:40)

ACK: 138 CHECKSUM: 65427

[Send 第 139 号数据包](2023-11-14 12:44:40)

[Recv 第139号数据包的ACK确认](2023-11-14 12:44:40)

ACK: 139 CHECKSUM: 65428

[Send 第 140 号数据包](2023-11-14 12:44:40)

丢包验证停等如下，延时设计为15s。以及重传触发

[Repeat-Send (Timeout) 140 号数据包]

[TIMEOUT]检测到超时，回到慢启动阶段
cwnd= 1 sstresh= 6

ACK: 69 CHECKSUM: 65463 THROUGHPUT: 101.000

[Send 第 77 号数据包]

ACK: 69 CHECKSUM: 65463 THROUGHPUT: 101.000

[Repeat-Send 70 号数据包]

[Repeat-Send 71 号数据包]

[Repeat-Send 72 号数据包]

[Repeat-Send 73 号数据包]

[Repeat-Send 74 号数据包]

[Repeat-Send 75 号数据包]

[Repeat-Send 76 号数据包]

[Repeat-Send 77 号数据包]

ACK: 69 CHECKSUM: 65463 THROUGHPUT: 101.000

ACK: 69 CHECKSUM: 65463 THROUGHPUT: 101.000

ACK: 69 CHECKSUM: 65463 THROUGHPUT: 101.000

握手成功:

服务器端

```
*****  
文件传输成功  
*****  
[wave success]挥手完成，成功断开连接
```

客户端



文件成功传输的资源管理器截图:



5. 实验问题解答

5.1 实验总结:

本次改进了上次停等机制实现中未完善的吞吐速率的日志以及精简了四次挥手的过程。同时测试出了发送 `helloworld.txt` 时候可能是由于纯文本头信息存在可能的 `curseq` 未被正确初始化为0的 `bug`，已经测试后再三握手成功建立文件本地连接后初始化，修复了这个 `bug`。

相比停等机制，GBN协议采用累积确认抵消了意外阻塞后的延时，本质上是一种保持局部有序到全局有序的思想。

5.1.1 1. 滑动窗口：

5.1.1.1 改进点：

1. **流水线传输：** 滑动窗口允许发送方连续发送多个帧，而不必等待每个帧的确认。这样可以充分利用网络带宽，提高传输效率。
2. **减少等待时间：** 发送方不需要等待每个帧的确认，而是可以继续发送下一组帧。这减少了在传输过程中的等待时间。
3. **更高的吞吐量：** 滑动窗口允许多个帧同时在传输途中，从而提高了整体的吞吐量。

5.1.1.2 缺点：

1. **复杂性增加：** 实现滑动窗口协议需要更复杂的逻辑，包括序号管理、窗口滑动等，相对于停等机制而言，它的实现较为复杂。
2. **需要更多的资源：** 滑动窗口需要维护窗口大小和序号，这可能需要更多的内存和处理资源。

5.1.2 2. 累积确认：

5.1.2.1 改进点：

1. **减少确认帧数量：** 累积确认允许接收方一次性确认多个已经成功接收的帧，而不是每个帧都发一个确认。这减少了确认帧的数量，减小了网络开销。
2. **提高效率：** 发送方可以在收到一个确认之前继续发送多个帧，而不必等待每个帧的确认。这提高了传输效率。
3. **降低延迟：** 由于累积确认可以一次性确认多个帧，因此在高延迟的网络中，它可以减少确认的往返时间。

5.1.2.2 缺点：

1. **失序的处理：** 累积确认可能导致失序的问题，即确认帧可能确认了接收方未按顺序接收的帧。因此，需要在协议中增加处理失序的机制。

5.2 实验不足

1.没有正确回答累积确认的概念，累积确认是将最大有序ACK接受后维护更新前面部分有序和失序的状态为有序。比如3-ACK未验证丢包，4-ACK未接收，继续接收5-ACK后。应该确认5-ACK并且将3、4的状态也更新为累积确认的接收状态，这样就需要单独开启多线程分别实现对计时器的计数。

2没有在分组后使用多线程进行发包。没有正确实现多线程的安全机制导致不能使用多线程编程技术准确地验证GBN算法。

将在SR算法中更改多线程的逻辑并且修改。